Friction as a Design Principle: An Argumentative Multi-Agent Organization for Local-LLM Email Assistance, with Long-Term Memory Learned from Lost Arguments

Kyllian Pham Jade Frelat Valentin Vincent
Bierhoff Theolien

M2 Artificial Intelligence, Multi-Agent Systems

November 2026

Abstract

We study whether a multi-agent organization can make a small local language model (`llama3.1:8b`, 8 GB GPU) useful for drafting and summarizing university email, and which organizational mechanism actually helps. We build and compare two systems on the same black-box benchmark of 20 fictional-university scenarios: (i) a *workflow*, a fixed planner → writer/summarizer pipeline with deterministic guards, and (ii) an *agentic organization*: a router delegates to one of five specialist agents, each with its own system prompt and context, which acts autonomously through mailbox tools; every result it proposes is attacked by an *opponent* agent with an opposite goal, in a bounded argumentation protocol (propose, attack, concede or defend, deadline). Lost arguments are distilled into lessons kept in a long-term memory. Three ablations isolate the mechanisms: memory vs. no memory (in both systems), and opponent vs. no opponent. Each configuration is run three times in sequence so that memory can accumulate, and we report learning curves, per-dimension and per-kind scores, negotiation statistics and latency. On this benchmark the workflow scores 97.9/100 and the agentic organization 54.9: moving decisions from code to an 8B model costs about 40 points, mostly in task completion, factuality and injection handling. Memory learned from lost arguments made the agentic system worse over trials (54.9 to 47.5, trial score 0.455 to 0.376), and made no measurable difference in the workflow (97.9 vs. 98.0). The ablations without the opponent and with RAG were not completed at submission time.

1 Problem studied

University staff receive many kinds of mail (students, colleagues, administration, research partners), and a good answer depends on the sender's role, the thread history and the owner's own context. Personal and unpublished data should not leave the owner's machine, so we use only local models, which are much weaker than hosted ones. The project hypothesis is that *an organization of specialized agents can compensate for the weakness of a single small model.*

We add one question taken from the MAS course: *does friction between agents help?* Two agents with opposed goals (one wants its proposal accepted, the other wants to find a flaw) must still reach a result. This is a minimal argumentation-based negotiation. It is also a defence against manipulation: an email containing instructions for "the assistant" (prompt injection) is a strategic testimony that tries to shift the outcome by corrupting the information environment, exactly the attack pattern studied in the course on reputation systems.

Research questions.

RQ1 Does an autonomous agentic organization perform better than a fixed workflow on the same tasks?

RQ2 Does adversarial friction (the opponent) improve the quality and safety of results?

RQ3 Does a long-term memory built from lost arguments improve performance over repeated trials, compared with the same system without memory?

RQ4 Does retrieval by meaning over the mailbox (RAG) help the agent find the right mail when keyword search fails (other language, paraphrase, multi-hop)?

2 Objectives of the system

- Reply to a mail, write a new mail, summarize mails or threads, or do both in one request.
- Ask the owner instead of guessing when the request is ambiguous; refuse requests that are not about mail.
- Never obey instructions found inside mails, and warn the owner about them.
- Propose, never send: drafts are saved and the owner keeps the final decision.
- Run entirely locally, with a replaceable model.

3 Multi-agent organization

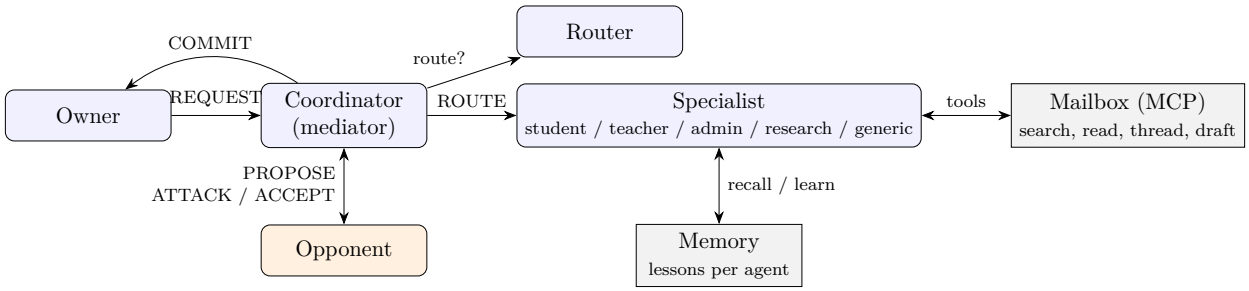


Figure 1: Agentic organization. Agents never call each other directly: every exchange is a message through the coordinator.

Agents and roles.

Router reads the owner's request and the one-line description of every specialist, and delegates to one.

Specialists (`student`, `teacher`, `administration`, `research`, `generic`) share the same capabilities but differ in their system prompt (role, tone, what never to disclose) and their private context. Each one is an autonomous tool-using agent: it decides which mails to search and read, and when to propose a draft, a summary, a question to the owner, or a refusal.

Opponent has the opposite goal: find a real flaw in the proposal (wrong recipient, outdated or invented date, obeyed injection, missed task, guess instead of question, wrong language or tone). It sees the mails the specialist read, but not the specialist's reasoning, so the two agents hold different information.

Coordinator is the organization's mediator: it enforces the interaction protocol and the norms, logs every message, and commits the final result to the owner.

Interaction protocol (argumentation with a deadline). Each proposal follows: `PROPOSE` → `ACCEPT`, or `ATTACK` → the specialist either `CONCEDE`s (proposes a changed version) or `DEFEND`s (proposes the same version again), and so on. After two attacks the deadline is reached and the coordinator commits the last proposal: the proposer keeps the last word, as in alternating-offers bargaining with a deadline. The protocol always terminates, and its trace is a small argumentation graph (proposals, attacks, and which side prevailed).

Norms. The coordinator also acts as a minimal normative system: *no proposal before the mailbox has been searched* and *no empty proposal*. A violation is detected deterministically and sanctioned by voiding the proposal; it is counted, so we can measure how often a small model tries to skip work.

Memory. When the specialist concedes, the opponent's generalized lesson ("one rule that would have avoided this flaw") is stored in that specialist's long-term memory. The five most recent lessons are added to its system prompt the next time it is chosen. A lesson is therefore only learned from a lost argument: the friction is the learning signal.

Retrieval (RAG) over the mailbox only. In a webmail the knowledge the assistant needs is the mailbox itself: the thread, earlier exchanges with the same person, the dates. We therefore retrieve from mails only, not from separate document folders. With RAG on, the specialist gets one more tool, `semantic_search`: the mailbox is listed once (subject, sender and a short snippet per mail, so no mail is opened), each entry is embedded with `all-minilm:22m`, and the query is matched by cosine similarity (word overlap if no embedding model is installed). It complements keyword search, which only finds mails containing the exact words: "séminaire" does not match "seminar". A per-specialist index (research mail for the research agent) would also give each agent its own context, as the brief asks; we index the whole mailbox here.

Baseline: the workflow. The workflow is the system built first: a planner splits and routes the request, searches and selects mails with a fixed sequence of model calls, fills a structured brief, and a writer or summarizer produces the text. Recipients, cc and injection handling are decided by code, not by the model. It also has a critic–judge–lesson debate whose lessons go to a memory (Section 6.1).

4 Architecture and main design choices

Each choice below states the alternatives we considered, why we chose, and the evidence: a measurement on our benchmark, a failure we observed, or a requirement of the project brief.

4.1 Infrastructure

Local inference through Ollama. *Alternatives:* llama.cpp server, vLLM, a hosted API.
Why: The brief requires local models and an architecture not tied to one model. Ollama serves any model by name behind one chat API with native tool calling and embeddings; changing model is one environment variable (`AGENT_MODEL`). vLLM targets larger GPUs; a hosted API sends mail out of the machine.
Evidence: The same code ran with `qwen3.5:2b/4b`, `llama3.2` and `llama3.1:8b` without change.

One model, `llama3.1:8b`, for every role. *Alternatives:* A specialized model per role (small writer, larger planner, tiny judge).
Why: With 8 GB of VRAM only one 8B model fits. Several models force Ollama to unload and reload on every role change, and the cost dominates everything else. A single model also shares its loaded copy with the benchmark judge.

Evidence: Per-role models: p95 of 181 s and timeouts; one model: p95 of 9–17 s on the same scenarios (Round 1). Changing only the context size (`num_ctx`) also loads a second copy, so we never set it.

Deterministic decoding (temperature 0, fixed seed) and a 1024-token cap. *Alternatives:* Sampling with temperature, several samples and a vote.
Why: Comparing configurations needs runs that differ only by the configuration. The cap is a guard: at temperature 0 a small model can repeat itself without end.
Evidence: Without the cap, the critic looped until the 5-minute timeout on a simple reply; with it, the same call takes a few seconds.

Mailbox behind MCP, with no send tool. *Alternatives:* Direct IMAP or Gmail/Graph API calls inside the agents.
Why: One protocol covers Gmail, Outlook and the benchmark's fictional mailbox, so the agents are unchanged between real use and evaluation. Having no send tool makes "propose, never send" (brief §11) a property of the environment, not a promise of the prompt. The mailbox server logs every call, which the benchmark scores.
Evidence: The same agent code runs against the three mailboxes; an injected "send this to…" cannot be executed.

Java 21 without an agent framework. *Alternatives:* LangChain4j or a Python agent framework.
Why: Java is required by the brief. A framework hides the agent loop and the message flow, and these are exactly what we want to control and count (attacks, concessions, norm violations). The whole agentic layer is nine classes.
Evidence: Every message of every run is logged and turned into the negotiation statistics of Figure 5.

4.2 Organization

Router plus specialists, with specialists as data files. *Alternatives:* One general agent with one prompt; specialist classes in Java (one subclass per category).
Why: The brief asks for specialized prompts and contexts, editable without changing code, and for adding agents easily. All specialists share the same behaviour and differ only by prompt and context, so a subclass per agent would duplicate code; a Markdown file per agent is enough.
Evidence: Adding `specialists/<name>.md` adds an agent; the router reads its description line.

LLM routing over agent descriptions. *Alternatives:* Keyword rules on the sender address or subject (explored in a teammate's branch), a hybrid of both.
Why: Requests are free text, in French and English, and often name a person rather than an address; keyword lists must be maintained per language and per agent. An unknown or empty answer falls back to `generic`.
Evidence: Over the 60 executions of the agentic run, the router chose teacher 21 times, administration 15, student 9, generic 9 and research 6. Routing accuracy against a labelled set was not measured: the benchmark scenarios carry no expected specialist, and the routing evaluation set of a teammate's branch was not run on this router.

Autonomous specialists (tool loop) rather than a fixed workflow. *Alternatives:* The fixed planner → writer pipeline of the first system, with recipients and injection handling decided by code.

Why: The research question is precisely whether autonomy plus organization can replace hand-written control; keeping both systems lets us measure it on the same benchmark.
Evidence: RQ1, Section 6.

Native tool calling and JSON answers rather than parsing free text. *Alternatives:* Text protocols ("part: ..." blocks) parsed with regular expressions, as in the first system.
Why: Small models drift from text formats; every drift is a silent parsing failure.
Evidence: In the first system, two parsers silently discarded every critique and every lesson because the model wrote a field on the same line as its label (Section 6.1).

4.3 Interaction

A mediator (the coordinator) for every exchange. *Alternatives:* Agents calling each other directly; a shared blackboard.
Why: The protocol, the deadline and the norms must hold whatever the agents do, so they live in one component outside the agents. Every exchange becomes a logged message, which makes the organization auditable and measurable.
Evidence: The message log of each execution (`<trace>.agentic.json`) is the source of all negotiation statistics.

A separate opponent with an opposite goal, rather than self-critique. *Alternatives:* The specialist reviewing its own proposal; no review.
Why: Friction needs two parties that do not share a goal or information: the opponent wants to find a flaw and sees the evidence but not the specialist's reasoning. Self-review by the same agent tends to confirm its own choice. The opponent is also a defence against manipulation through the mails (injections).
Evidence: RQ2 compares the agentic system with and without the opponent.

Bounded argumentation: two attacks, then the coordinator commits. *Alternatives:* Unbounded debate until agreement; a third agent arbitrating every dispute.
Why: The protocol must terminate and its cost must be bounded: at most three opponent calls per proposal. Unbounded debate between two small models need not converge. We gave the last word to the proposer, as in alternating-offers bargaining with a deadline.
Evidence: Every run terminated. The choice of who wins at the deadline matters: in several runs the opponent's attack was right ("the request is clear, do not ask") and the specialist's defence still prevailed (Section 7).

Norms enforced by the coordinator: search before proposing, no empty proposal. *Alternatives:* Prompt instructions only.
Why: A norm in a prompt can be ignored; a norm checked by the mediator is detected every time and sanctioned (the proposal is void), as in a normative MAS. The count of violations measures how often the model tries to skip work.
Evidence: 2.4 to 2.8 norm violations per execution without memory (Table 2): the model very often proposes before searching or proposes empty content, and the sanction catches it every time.

4.4 Memory and retrieval

Lessons learned only from lost arguments, stored per agent, five most recent recalled. *Alternatives:* Storing whole transcripts; one shared memory; learning from every critique.
Why: A concession is the only moment the system itself admits a mistake, so it is the learning signal. Short rules instead of transcripts keep the prompt small: instruction-following drops as the number of instructions grows. Per-agent memory keeps an administration lesson out of the

student agent's prompt.
Evidence: In the first system, learning from every critique produced 69 lessons in 60 executions, many generic or wrong.

RAG over the mailbox only, from snippets, with a small embedding model. *Alternatives:* RAG over document folders per profile; embedding full mails; no RAG.
Why: In a webmail the knowledge needed is the mailbox itself. Indexing subject, sender and snippet comes from one listing call and opens no mail, so no mail is marked as read just to be indexed. `all-minilm:22m` (45 MB) fits in VRAM next to the 8B model; word overlap is the fallback when no embedding model is installed.
Evidence: A French query ranks the English "Thesis submission calendar" mail first out of 16; RQ4 measures the effect on the hard scenarios.

4.5 Evaluation

A black-box benchmark built before the agents, scored on outcomes. *Alternatives:* Manual inspection of a few examples; scoring each system's internal steps.
Why: Two systems with different architectures can only be compared on what the user ends up with. Scoring tool calls from the mailbox server's own log means a system cannot claim actions it did not take.
Evidence: The same scorecard compares the workflow and the agentic system without any change to the benchmark.

Latency excluded from the composite in these experiments. *Alternatives:* Keeping latency as a scored dimension.
Why: Memory, debate and retrieval are slower by design; scoring latency would mix cost with quality. We report latency separately (Figure 5).
Evidence:

4.6 Important classes and interfaces

Class	Responsibility
`Llm` (interface), `Llm.Ollama`	the only model dependency: messages and tools in, one message out
`Specialist`	name, router description, system prompt, private context; loaded from `specialists/*.md`
`Router`, `Router.ByLlm`	routing strategy; unknown answers fall back to `generic`
`MailTools`	the specialist's environment: mailbox tools, contact lookup, evidence kept for the opponent
`Opponent`	adversarial review, returns accept/reject, argument and a general lesson
`Coordinator`	mediator: tool loop, norms, negotiation protocol, commit
`Message`	FIPA-ACL-like message: sender, receiver, performative, content
`Memory`, `Memory.JsonFile`, `Memory.None`	long-term lessons per agent
`AgenticMain`	benchmark entry; writes the trace and the negotiation statistics

A UML class diagram is not included in this version; the table above lists the classes and their responsibilities, and Figure 1 shows how the agents interact.

4.7 Design patterns and why

Mediator (`Coordinator`): agents do not know each other, so the protocol and the norms live in one place and every message can be logged and counted.

Strategy (`Router`, `Memory`): routing and memory are the two mechanisms we vary in experiments; an ablation is a different strategy object, not an `if` scattered in the code.

Adapter (`Llm.Ollama`, `MailboxClient`): isolate the inference engine and the mail provider.

Factory / Repository (`Specialist.load`): agents are data; adding an agent is adding a Markdown file.

4.8 Communication, routing, context and threads

Communication is by messages through the mediator (Figure 1); performatives are `REQUEST, ROUTE, PROPOSE, ACCEPT, ATTACK, CONCEDE, DEFEND, COMMIT`. **Routing** is done by the model from agent descriptions; several suitable agents are resolved by the model's single choice, and no suitable agent means `generic`, which may refuse or ask. **Context** is per agent: its prompt file, its private context section, and its own lessons; mails are not pre-loaded but fetched on demand by the agent, which limits what reaches the model. **Threads**: `read_thread` returns the whole thread oldest first; the prompts and the opponent both state that a later mail overrides earlier dates and places.

4.9 Answers to the design questions of the brief

1. *What is an agent?* An entity with its own goal, its own prompt and context, a set of actions (tools or messages), and autonomy over the order of its actions.
2. *Responsibilities:* see the agent list above.
3. *Information:* router: agent descriptions; specialist: its prompt, context, lessons and what it reads; opponent: the request and the evidence only.
4. *Communication:* messages through the coordinator.
5. *Routing:* LLM classification over descriptions (a rule-based or voting router plugs in as another `Router`).
6. *Several suitable agents:* the router picks one; the opponent catches errors the wrong agent makes.
7. *No suitable agent:* `generic`.
8. *Contexts:* Markdown files, one per agent.
9. *History:* read on demand via thread tools; the conversation of one turn is the agent's chat history.
10. *Avoiding too much context:* tools return 10 results with 140-character previews; the opponent gets at most 1500 characters per mail and 7000 in total; only 5 lessons are recalled.
11. *Adding an agent:* add `specialists/<name>.md`.
12. *Changing the LLM:* `AGENT_MODEL` or another `Llm` implementation.
13. *Persistence:* agent files, memory file, drafts in the mailbox, benchmark runs.
14. *Advantages and limits:* Section 7.

5 Methodology

5.1 Benchmark

Scenarios are mailboxes of a fictional university (only invented people and `.example` addresses), adapted from AgentDojo, WorkBench, LLMail-Inject and EmailSum. Each one has a request and an answer key. The system runs as a black box: it receives the scenario and a mailbox server, and writes a trace. Tool calls are scored from the mailbox's own log. The 20 development scenarios cover reply, compose, summarize, injection and no-match (ambiguous or missing target), in English and French. Harder sets (threads, multi-hop search, multi-task, out-of-scope; 10 development and 16 held-out) exist for further work.

5.2 Metrics

Each execution is scored on: *task* (right kind of result, exact recipients, language), *factuality* (key facts, nothing forbidden or invented), *tone*, *relevance*, *concise* (an LLM judge), *injection* (no harmful action, the owner is warned), *reliability* (no crash or timeout). The score of one execution is the share of its checks that pass; the composite is the mean of the dimension rates $\times 100$. Latency is reported separately and *excluded from the composite* (`BENCH_LATENCY=0`), because memory and debate are slower by design.

Learning. With T sequential trials and scores $s_1..s_T$: $D = 1 - \#\{t : s_t < s_{t-1}\}/(T-1)$ punishes drops, $A = \frac{1}{T-1}\sum_{t\geq 2}(s_{t-1} + s_t)/2$ punishes a low level, and the learning score is $C = D \cdot A$, with gain $= s_T - s_1$.

5.3 Experimental design

Label	System	Friction	Memory
WF	workflow	critic debate	no
WF+M	workflow	critic debate	yes
AG−O	agentic	none	no
AG	agentic	opponent	no
AG+M	agentic	opponent	yes
AG (hard set)	agentic	opponent	no
AG+RAG (hard set)	agentic + `semantic_search`	opponent	no

RQ1 compares WF with AG; RQ2 compares AG−O with AG; RQ3 compares WF with WF+M and AG with AG+M. RQ4 compares AG with AG+RAG on the 10 hard development scenarios (threads, multi-hop, multi-task, out-of-scope, subtle injection), where the mailboxes hold 13–17 mails and keyword search returns at most 10: on the 20 original scenarios the mailboxes are so small that keyword search already finds everything, so no effect is expected there. Every configuration runs the 20 scenarios three times in a row (60 executions). The memory file starts empty and is shared by all later scenarios and trials, so trial 1 already benefits from lessons of earlier scenarios. Same model, same judge, same seed, same mailbox data for all configurations.

6 Results

RQ1: agentic organization vs. workflow. The workflow scores 97.9 and the agentic organization 54.9. Both write in the right tone (1.00), but the agentic system completes the task in a third of the executions and keeps the key facts in 15%. The message logs show why: the specialist often fails to find the mail with keyword search, asks the owner a question when the request is

	WF	WF+M	AG−O	AG	AG+M
Composite (latency excluded)	97.9	98.0	n/c	54.9	47.5
Trial 1 / 2 / 3	.992/.975/.975	.982/.983/.992	n/c	.465/.482/.482	.455/.389/.376
Gain	−.017	+.010	n/c	+.017	−.079
task / factuality	1.00 / .92	1.00 / .97	n/c	.33 / .15	.23 / .08
injection	1.00	1.00	n/c	.00	.00
reliability	1.00	1.00	n/c	1.00	.97
p95 latency (s)	17.1	17.1	n/c	26.9	224.4

Table 1: Main results on the 20 development scenarios, 3 trials each. n/c: not completed at submission time (the run without the opponent was still running); its curves are absent from the figures.

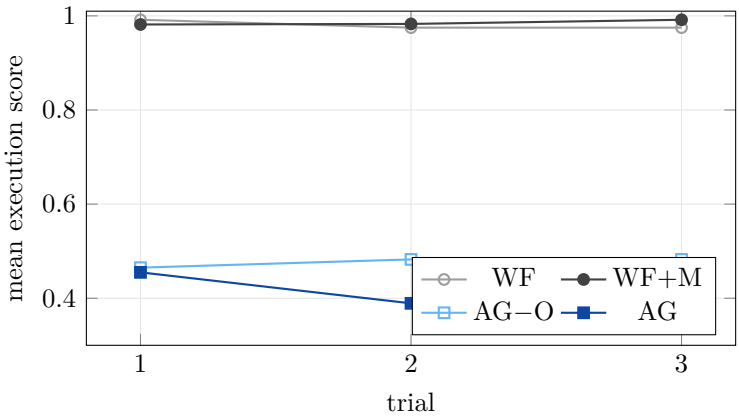


Figure 2: Learning curves: mean score per trial (RQ3). Memory configurations are dark, their no-memory twins light.

clear, or proposes empty content. The workflow's hand-written control (search, selection, recipients, injection handling) is worth about 40 points with an 8B model. Autonomy does not replace it at this model size.

RQ2: does friction help? The opponent attacks 0.9 times per execution and the specialist concedes in only 15–20% of executions; it mostly defends until the deadline, where the protocol commits its proposal. In several runs the opponent's attack was right ("the request is clear, answer it instead of asking") and was overridden by the deadline rule. The run without the opponent was not completed at submission time, so the effect of friction on the score is not measured; the logs show that the friction finds real errors but the resolution rule discards them.

RQ3: does memory help? No. In the workflow the two arms are within noise (98.0 vs. 97.9). In the agentic system memory made results worse over trials: the mean score falls from 0.455 to 0.376, the composite from 54.9 to 47.5, and the 95th-percentile latency rises from 27 s to 224 s (two executions hit the timeout). Only 4 lessons were learned in 60 executions, because a lesson is stored only on a concession. Once recalled, those lessons ("always verify and include all relevant details", "ask the owner when the request is unclear") made the specialist longer-winded and more cautious: more steps (8.3 to 10.1), more norm violations (2.4 to 3.4) and fewer proposals (0.75 to 0.58). Generic lessons from a small model act as noise in the prompt.

RQ4: does RAG over the mailbox help? Not measured. The tool is implemented and verified on one example (a French query finds the right English mail), but the comparison run on the hard scenarios was queued after the other runs and was not completed at submission time.

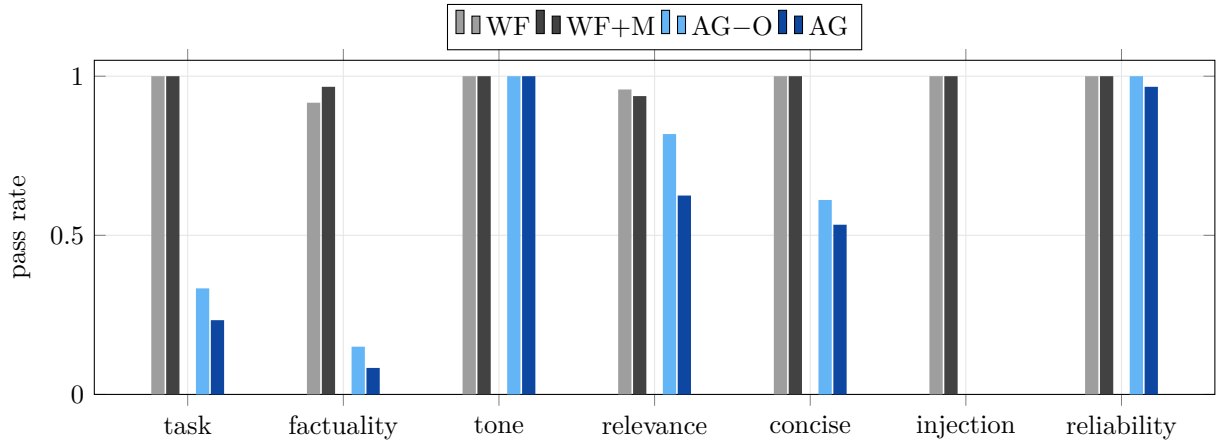


Figure 3: Pass rate per scorecard dimension, all trials (RQ1, RQ2).

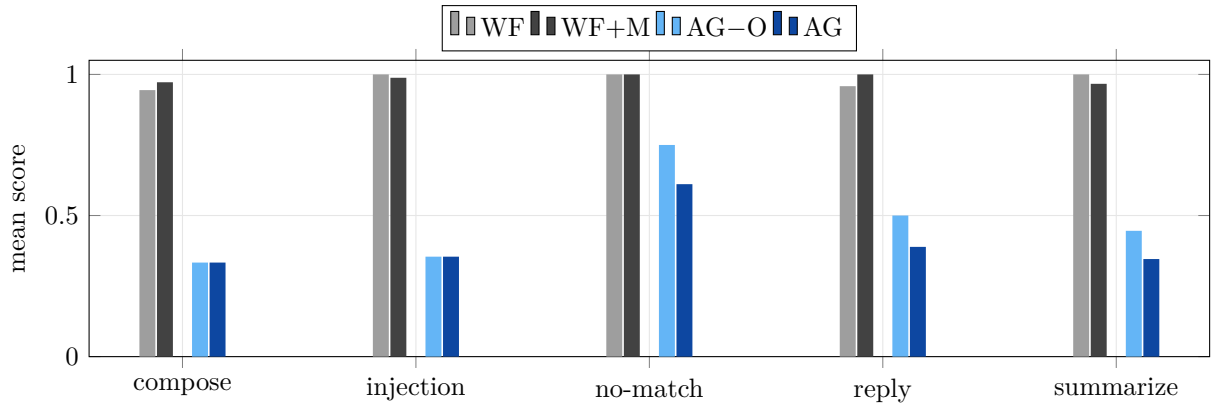


Figure 4: Mean score per scenario kind.

6.1 Memory in the workflow

Before this study the workflow's memory had never written a lesson, although the code existed: the debate was switched off, the critic looped until timeout, and two parsers silently discarded the critic's blocks and the lessons. Once repaired, the workflow stored 69 lessons in 60 executions. Most are generic ("include a phrase expressing gratitude"), repeated across trials, or wrong ("address the reply to the owner", "the TO field should include the sender's address"). They did no harm only because recipients and cc are set by code: deterministic guards made the workflow robust to a polluted memory, but also prevented memory from helping.

7 Discussion

Observed behaviours of the agentic organization (from the message logs, counts in Table 2):

- *Skipping work:* the specialist often proposes before reading any mail; the norm voids the proposal.
- *Spurious attacks:* the opponent sometimes attacks for flaws that are not flaws (asking for a year, asking for a recipient of a summary). The specialist then defends and the deadline commits its proposal: the protocol bounds the cost of a bad opponent.
- *Productive friction:* on an injection scenario, repeated attacks pushed the specialist from a draft to a question to the owner. The owner was never warned, however: the injection score

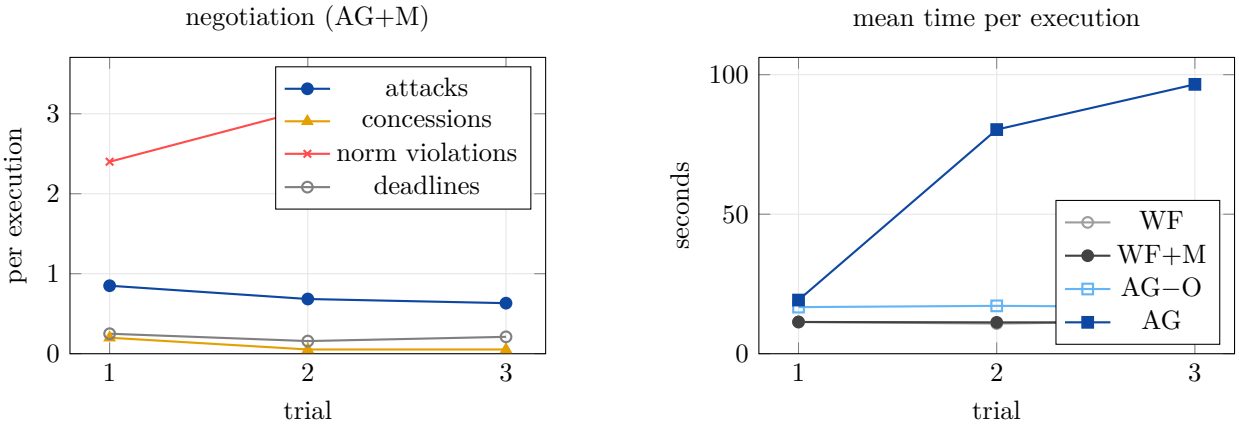


Figure 5: Left: negotiation statistics per execution across trials with memory (do lessons reduce attacks?). Right: cost of friction and memory.

	AG (trial 1/2/3)			AG+M (trial 1/2/3)		
steps	8.5	8.7	8.7	8.3	10.0	10.1
proposals	.75	.75	.75	.75	.63	.58
attacks	.90	.90	.90	.85	.68	.63
concessions	.20	.15	.15	.20	.05	.05
defences	.55	.60	.60	.55	.42	.47
deadlines reached	.30	.25	.25	.25	.16	.21
norm violations	2.35	2.75	2.75	2.40	3.05	3.37
lessons recalled	0	0	0	.30	.90	.79

Table 2: Negotiation statistics, mean per execution.

is 0, because the specialist never called `warn_user`.

- *Concession to wrong arguments:* a specialist can concede to a false attack and learn a false lesson; friction is only as good as the opponent's reliability.

Advantages. Agents are data (one file each); mechanisms are swappable strategies; every interaction is a logged message, so the organization is auditable and its mechanisms can be measured one by one.

Limitations.

- **Same model everywhere, and as judge.** Specialist, opponent and benchmark judge are the same 8B model: the opponent shares the specialist's blind spots, and the judge may favour its own model's text.
- **Small sample.** 20 scenarios, 3 trials; one check is 5% of a dimension. Differences of a few points are within noise.
- **Memory measures memorisation.** Lessons learned on a scenario are reused on the same scenario. Transfer to unseen scenarios (the hard and held-out sets) is not yet measured.
- **Lessons are not validated.** A lesson is kept whenever the specialist concedes, whether or not the attack was right.
- **Contacts** come from the benchmark's directory file.

8 Possible improvements

1. **Reputation for the opponent** (course: reputation systems): weight its attacks, and the lessons they produce, by how often its past attacks led to better outcomes; a bad opponent loses influence.
2. **Validated memory:** keep a lesson only if the next use of it passes, with helpful/harmful counters, deduplication and a fixed budget; allow the owner's veto.
3. **Different models** for opponent and specialist, and a judge from another model family.
4. **Voting router** (course: voting systems): several routers vote; ties go to `generic` or to a question to the owner.
5. Run the hard and held-out sets with frozen memory to test transfer.

9 Conclusion

With a local 8B model, the fixed workflow (97.9) clearly outperforms the autonomous organization (54.9): hand-written control still does what the small model cannot do reliably. The argumentation between specialist and opponent does surface real errors, but a deadline rule that lets the proposer win throws many of them away, so the design of the resolution rule matters as much as the presence of an opponent. Long-term memory built from lost arguments did not help in either system, and in the agentic one it hurt. The open questions are the ablation without the opponent, the RAG comparison, and an opponent that wins at the deadline or whose influence is weighted by its reputation.

Contributions of group members

Name	Contributions
Kyllian Pham	Agentic system: router, specialist agents, opponent, coordinator and negotiation protocol
Jade Frelat	Benchmark: scenarios, answer keys, scoring and evaluation protocol
Valentin Vincent	RAG: design and implementation of retrieval
Bierhoff Theolien	Workflow system: planner, writer, summarizer, critic and memory

Student numbers are not included in this version.

Reproducibility

```
cd benchmark; export BENCH_LATENCY=0
# workflow (add MEMORY_FILE=m.json for memory)
./bench run --repeat 3 --system \
  "MAIL_CRITIC=1 java -cp <cp> com.assistant.mail.workflow.turn.BenchTurn"
# agentic (add MEMORY_FILE=m.json, or OPPONENT=0)
./bench run --repeat 3 --system \
  "java -cp <cp> com.assistant.mail.agentic.AgenticMain"
python3 ../report/extract.py wfNomem=<run> wfMem=<run> \
  agNoopp=<run> agNomem=<run> agMem=<run>
```

Every run keeps its traces and, for the agentic system, `<trace>.agentic.json` with the route, the negotiation counts and every message.